DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0607 Design and Analysis of Algorithms

ASSESSMENT – CO3

Brute-Force and Exhaustive-Search Techniques

Complete Assessment Report

Item	Details
Student name	RITHIKA G
Register number / Section	192571027
Course Outcome	**CO3- Module 4 – Recommendation Scores (Strassen's Matrix Multiplication)**
Nature / Marks	Individual · 50 marks
GitHub repository	https://github.com/192571027/CSA0607-
Implementation language	Python 3.12 (matplotlib for graphs)

Module 4 – Recommendation Scores (Strassen's Matrix Multiplication)

Case: the score of every product for every customer segment is C = A × B, where A is the segment–feature matrix and B the feature–product matrix. In production both are n × n with **n = 1,024**, and recommendations are refreshed only once a day.

Chosen algorithm: Hybrid Strassen (recursive down to a cut-off block, then an optimised conventional/BLAS kernel).

---

1. Divide, Conquer, Combine

Split each matrix into four n/2 × n/2 blocks:

A = | A11 A12 | B = | B11 B12 | C = | C11 C12 |
| A21 A22 | | B21 B22 | | C21 C22 |

Step	Conventional block method	Strassen
Divide	split into blocks	split into blocks, form 10 block sums/differences
Conquer	**8** recursive block products	**7** recursive block products M1…M7
Combine	4 block additions	8 block additions/subtractions

Saving one of the eight recursive multiplications is the whole idea.

---

2. The seven products and the reconstruction formulas

M1 = (A11 + A22)(B11 + B22)

M2 = (A21 + A22) B11

M3 = A11 (B12 − B22)

M4 = A22 (B21 − B11)

M5 = (A11 + A12) B22

M6 = (A21 − A11)(B11 + B12)

M7 = (A12 − A22)(B21 + B22)

C11 = M1 + M4 − M5 + M7

C12 = M3 + M5

C21 = M2 + M4

C22 = M1 − M2 + M3 + M6

Algebraic check of one entry: M1 + M4 − M5 + M7 = A11B11 + A11B22 + A22B11 + A22B22 + A22B21 − A22B11 − A11B22 − A12B22 + A12B21 + A12B22 − A22B21 − A22B22 = **A11B11 + A12B21 = C11** ✓.

---

3. Verification on the sample matrices

A = [[1, 7], [3, 5]], B = [[1, 3], [7, 5]] (A11 = 1, A12 = 7, A21 = 3, A22 = 5; B11 = 1, B12 = 3, B21 = 7, B22 = 5)

Product	Substitution	Value
M1	(1 + 5)(1 + 5) = 6 × 6	**36**
M2	(3 + 5) × 1 = 8 × 1	**8**
M3	1 × (3 − 5) = 1 × (−2)	**−2**
M4	5 × (7 − 1) = 5 × 6	**30**
M5	(1 + 7) × 5 = 8 × 5	**40**
M6	(3 − 1)(1 + 3) = 2 × 4	**8**
M7	(7 − 5)(7 + 5) = 2 × 12	**24**

Entry	Formula	Value	Conventional check
C11	M1 + M4 − M5 + M7 = 36 + 30 − 40 + 24	**50**	1·1 + 7·7 = 50 ✓
C12	M3 + M5 = −2 + 40	**38**	1·3 + 7·5 = 38 ✓
C21	M2 + M4 = 8 + 30	**38**	3·1 + 5·7 = 38 ✓
C22	M1 − M2 + M3 + M6 = 36 − 8 − 2 + 8	**34**	3·3 + 5·5 = 34 ✓

C = [[50, 38], [38, 34]] with **7** scalar multiplications instead of 8.

The recursive implementation was also tested against A @ B on random integer matrices of sizes 2, 4, 8, 16, 32, 64 and 100 (padded to 128): all results are identical, and the number of scalar multiplications is exactly 7^k (7, 49, 343, … 823,543).

4. Recurrence and complexity

Multiplications. $M(n) = 7\,M(n/2)$, $M(1) = 1 \Rightarrow$ **$M(n) = 7^{(\lg n)} = n^{(\lg 7)} \approx n^{2.807}$**.

Total running time. The 18 block additions of size $(n/2)^2$ give

$T(n) = 7\,T(n/2) + 18\,(n/2)^2 = 7\,T(n/2) + \Theta(n^2)$

Master theorem: $a = 7$, $b = 2$, $f(n) = \Theta(n^2)$; $n^{(\log_2 7)} = n^{2.807}$ and $f(n) = O(n^{(2.807 - \varepsilon)}) \Rightarrow$ case 1 $\Rightarrow$ **$T(n) = \Theta(n^{(\lg 7)}) \approx \Theta(n^{2.807})$**, compared with $\Theta(n^3)$ for the conventional method (recurrence $8T(n/2) + \Theta(n^2)$).

Exact additions. $A(n) = 7A(n/2) + 18(n/2)^2$, $A(1) = 0 \Rightarrow$ **$A(n) = 6\,(n^{(\lg 7)} - n^2)$**.

Space. The block sums and the seven products need temporary matrices: $\Theta(n^2)$ extra memory (about 17 half-size temporaries per level in a naive implementation).

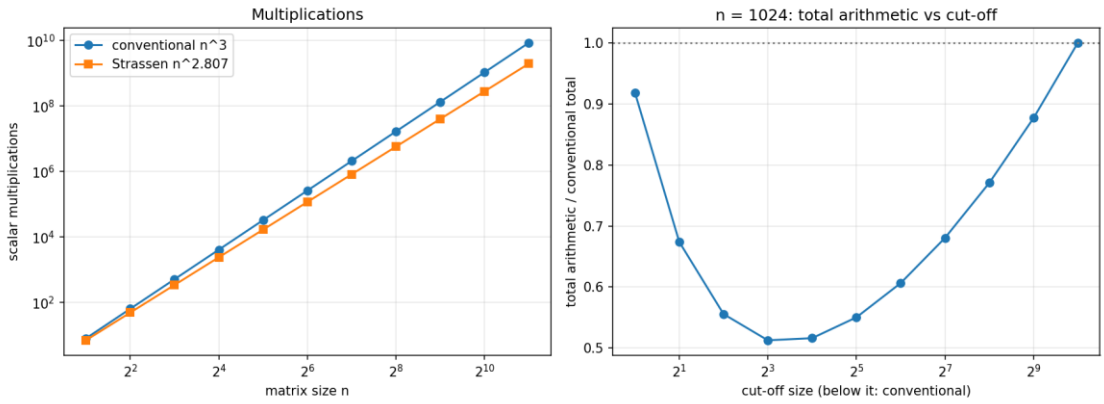
---

5. Comparison for n = 1,024 (= 2^{10})

Quantity	**Conventional**	**Strassen (recursing to 1 × 1)**
Scalar multiplications	$n^3 = 1{,}024^3 =$ **1,073,741,824**	$7^{10} =$ **282,475,249**
Ratio	1	0.2631 → **3.80 × fewer** (73.7 % saved)
Scalar additions	$n^2(n-1) = 1{,}072{,}693{,}248$	$6(7^{10} - 4^{10}) = 1{,}688{,}560{,}038$
Total arithmetic	2,146,435,072	1,971,035,287
Total ratio	1	0.9183 → only **8.2 % fewer**

Calculation: $7^{10} = 282{,}475{,}249$; $4^{10} = n^2 = 1{,}048{,}576$; difference 281,426,673; × 6 = 1,688,560,038.

So the headline "3.8× fewer multiplications" is misleading when counting *all* arithmetic: Strassen replaces multiplications by a large number of additions, and when both operations cost about the same (as in floating-point hardware) the saving is only about 8 %.



Strassen vs conventional

---

6. Why Strassen may still not be faster in practice

1. **Additions are not free.** 18 block additions per recursion level; the total arithmetic saving is only 8 % at full recursion (table above). One level of Strassen is cheaper than the conventional method *only for $n \geq 16$* (arithmetic count: n = 8 → 1,072 vs 960; n = 16 → 7,872 vs 7,936).
2. **Memory traffic and cache behaviour.** Many temporary matrices (≈ 45 MiB extra for n = 1,024 in the naive version versus 24 MiB for A, B, C) and repeated passes over the data; modern CPUs are limited by memory bandwidth rather than by multiplications.
3. **Highly optimised conventional kernels.** BLAS libraries use cache blocking, SIMD and fused multiply-add and reach close to peak machine speed on the n^3 algorithm.
4. **Recursion overhead** for small blocks (function calls, slicing).
5. **Numerical stability.** Strassen's error bounds are weaker than those of the conventional algorithm (measured max error vs BLAS for float64 grows with the number of levels: 9×10^{-13} at one level up to 6×10^{-12} at four levels in my test) – acceptable for ranking scores but worth knowing.
6. **Size restrictions.** n must be a power of two (zero padding otherwise) and sparse matrices lose their structure.

Measured on this machine (single core, numpy/BLAS leaves, float64, n = 1,024, best of 5)

Method	Time	Max error vs BLAS
Conventional (BLAS)	0.043 s	–
Hybrid Strassen, cut-off 512 (1 level)	0.052 s	9.1×10^{-13}
Hybrid Strassen, cut-off 256 (2 levels)	0.069 s	2.6×10^{-12}
Hybrid Strassen, cut-off 128 (3 levels)	0.077 s	3.6×10^{-12}

Method	Time	Max error vs BLAS
Hybrid Strassen, cut-off 64 (4 levels)	0.110 s	6.0×10^{-12}

Every Strassen variant is *slower* than the BLAS kernel here, and the more levels it recurses, the slower it gets. The same experiment with pure-Python leaves at n = 128 also showed the hybrid slower than the conventional method for every cut-off tested (ratio 1.2–2.0). These timings are specific to this machine and my Python/numpy implementation (absolute numbers will differ on yours), but the trend – a fast conventional kernel beats deep Strassen recursion at n = 1,024 – is the lesson.

---

7. Practical hybrid

Recommendation: use Strassen only for the top levels, and an optimised conventional kernel below a cut-off.

- **Cut-off block size.** By pure operation count the best cut-off for n = 1,024 is **8** (1,100,550,336 total operations, 48.7 % fewer than conventional); the table shows that any cut-off from 8 to 64 stays below 0.61× the conventional count:

Cut-off	Levels of Strassen	Multiplications	Total arithmetic	vs conventional
1	10	282,475,249	1,971,035,287	0.918
8	7	421,654,016	1,100,550,336	0.513
16	6	481,890,304	1,108,079,872	0.516
64	4	629,407,744	1,301,696,512	0.606
128	3	719,323,136	1,460,453,376	0.680
256	2	822,083,584	1,653,932,032	0.771
512	1	939,524,096	1,881,931,776	0.877
1024	0	1,073,741,824	2,146,435,072	1.000

- **In practice the cut-off should be chosen by benchmarking, typically 128–512 on a machine with an optimised BLAS**, because memory and cache effects (points 2–3 above) move the optimum far above the operation-count optimum of 8. Use the benchmark to decide whether to use Strassen at all: on my machine the answer for n = 1,024 was *no*, and a hybrid with 1 level was the closest competitor.
- Switch to the conventional kernel automatically below the cut-off; pad n up to a power of two (or peel odd rows/columns).

- Allocate temporaries once and reuse them (about n^2 extra memory instead of re-allocating at each call); run the seven products on different cores (they are independent).
- Because recommendation scores are only used for ranking, the slightly larger floating-point error of Strassen is acceptable; for exact or financial results use the conventional kernel.
- Since the refresh happens once per day, the real business gain comes from *running the product more often* (e.g. hourly) using the fastest kernel, GPU or incremental updates (C changes only where A or B changed), more than from the choice of Strassen itself.